

A COOKBOOK FOR DEVELOPERS

# Fine-Tuning Cookbook

Adapt a model to your task — when it's worth it (rarely), and how to do it well with LoRA/QLoRA, SFT, and DPO — for a developer who already prompts and builds with LLMs.

LoRA / QLoRA

PEFT

SFT

DPO / RLHF

Hugging Face / TRL

Data prep

Evaluation

Serving

---

Written for a developer who can already prompt, do RAG, and build agents, and now wants to change the model's weights — not just its context.

Every chapter answers: what it is, how to do it in Python, the pitfalls, and when prompting or RAG is the better tool (usually).

Edition 2026 · the dataset is the product · adapt, don't retrain

# Table of Contents

---

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| <b>How to Use This Book</b>                                     | <b>4</b>  |
| The one idea: change the weights, not just the context.         | 4         |
| What this book covers.                                          | 5         |
| <b>Part 1 · When to Fine-Tune (and When Not)</b>                | <b>6</b>  |
| 1.1 The four levers, in order of cost                           | 6         |
| 1.2 The question that settles it: knowledge or behaviour?       | 6         |
| 1.3 Good reasons to fine-tune                                   | 7         |
| 1.4 When NOT to fine-tune                                       | 7         |
| <b>Part 2 · The Landscape: Full, PEFT, SFT &amp; Preference</b> | <b>8</b>  |
| 2.1 Two independent choices                                     | 8         |
| 2.2 The typical pipeline                                        | 8         |
| 2.3 Open vs hosted fine-tuning                                  | 9         |
| <b>Part 3 · Data Preparation — the Dataset Is the Product</b>   | <b>10</b> |
| 3.1 The format: chat / instruction pairs                        | 10        |
| 3.2 Quality beats quantity                                      | 10        |
| 3.3 Splits and leakage                                          | 11        |
| <b>Part 4 · Supervised Fine-Tuning (SFT)</b>                    | <b>12</b> |
| 4.1 What SFT actually does                                      | 12        |
| 4.2 The minimal SFT run (TRL)                                   | 12        |
| 4.3 The knobs that matter.                                      | 13        |
| 4.4 Watch the loss, but trust the eval                          | 13        |
| <b>Part 5 · LoRA &amp; QLoRA — Fine-Tuning You Can Afford.</b>  | <b>14</b> |
| 5.1 The LoRA idea                                               | 14        |
| 5.2 QLoRA: LoRA on a quantized base                             | 14        |
| 5.3 LoRA/QLoRA in code (peft + trl)                             | 14        |
| 5.4 Adapters are swappable                                      | 15        |
| <b>Part 6 · Preference Tuning (DPO &amp; RLHF).</b>             | <b>16</b> |
| 6.1 SFT vs preference — the difference                          | 16        |
| 6.2 RLHF vs DPO.                                                | 16        |
| 6.3 The usual order: SFT first, then DPO.                       | 17        |
| <b>Part 7 · Evaluating a Fine-Tuned Model</b>                   | <b>18</b> |
| 7.1 Always compare against the baseline                         | 18        |
| 7.2 What to measure                                             | 18        |
| 7.3 How to score.                                               | 19        |
| <b>Part 8 · Serving a Fine-Tuned Model</b>                      | <b>20</b> |
| 8.1 Adapter: load-on-top or merge                               | 20        |
| 8.2 Use a real inference engine                                 | 20        |
| 8.3 Quantize for inference too.                                 | 21        |
| <b>Part 9 · Fine-Tuning in Production</b>                       | <b>22</b> |
| 9.1 The real cost is iteration and maintenance                  | 22        |

|                                                       |           |
|-------------------------------------------------------|-----------|
| 9.2 Version everything, reproducibly . . . . .        | 22        |
| 9.3 The iteration loop . . . . .                      | 22        |
| 9.4 Base models move under you . . . . .              | 23        |
| <b>Part 10 · Capstone &amp; Cheat Sheet . . . . .</b> | <b>24</b> |
| 10.1 Capstone — fine-tune a model, honestly. . . . .  | 24        |
| 10.2 Decision cheat sheet. . . . .                    | 24        |
| 10.3 Concept cheat sheet . . . . .                    | 24        |
| 10.4 The five rules. . . . .                          | 25        |

# How to Use This Book

You can already steer an LLM three ways from the other books: **prompting** (instructions and examples in the context), **RAG** (retrieving facts into the context), and **agents** (giving it tools). This book is the fourth, heaviest lever: **fine-tuning** — actually changing the model's weights by training it on your own examples. It's the most powerful way to change a model's behaviour, and the one you should reach for *least*.

The honest headline, up front: **most teams who think they need fine-tuning don't**. Prompting and RAG solve the large majority of "make the model do X" problems faster, cheaper, and without a training pipeline to maintain. This book teaches fine-tuning properly *and* teaches you to recognise the minority of cases where it's genuinely the right tool.

Each chapter answers four questions:

1. **What is this and when do you use it?**
2. **How do you do it in Python?**
3. **What are the pitfalls?**
4. **When is prompting or RAG the better call?** (Usually.)

## The one idea: change the weights, not just the context

Prompting and RAG change what the model *sees* at inference time; the model itself is frozen. Fine-tuning changes the model *itself* — you continue training it on examples of the behaviour you want, so the new behaviour is baked into the weights.

### DIAGRAM

|                  |                  |                                   |
|------------------|------------------|-----------------------------------|
| Prompting / RAG: | [ frozen model ] | ← you craft the INPUT (context)   |
| Fine-tuning:     | [ new weights ]  | ← you change the MODEL (training) |

That power is also the cost: fine-tuning needs a curated dataset, a training run, evaluation, versioning, and serving infrastructure — a whole ML pipeline — where a better prompt needs none of it.

### ENGINEER'S MENTAL MODEL

Prompting is configuration; RAG is giving the program data at runtime; fine-tuning is **recompiling the binary**. It's the most invasive change, so it's the last one you reach for — after you've confirmed configuration and runtime data can't get you there. And modern fine-tuning rarely retrains the whole model: with LoRA (Part 5) you train a tiny adapter and leave the base weights untouched — more like a patch than a rebuild.

## What this book covers

The decision (when to fine-tune vs prompt vs RAG), the landscape (full vs parameter-efficient; supervised vs preference tuning), **data preparation** (the part that actually determines success), **supervised fine-tuning (SFT)**, **LoRA/QLoRA** (the practical default), **preference tuning (DPO/RLHF)**, **evaluation**, **serving** fine-tuned models, and the **production** realities of cost and iteration. It uses the Hugging Face ecosystem (`transformers`, `peft`, `trl`).

### HOW TO READ IT

Read Parts 1-2 before anything else — the decision and the landscape save you from fine-tuning something you shouldn't. Then Parts 3-5 are the core path most people actually take (data → SFT → LoRA). Part 6 (preference tuning) is for when SFT isn't enough; Parts 7-9 make it real. If you finish Part 1 and conclude you don't need to fine-tune, the book did its job.

### READ THIS FIRST: TRY THE CHEAPER LEVERS

Before fine-tuning, exhaust: a **better prompt** (clear instructions, few-shot examples, output schema), **RAG** (if the gap is missing *knowledge*), and a **bigger/better base model**. These are faster to try, need no training pipeline, and fix most problems. Fine-tuning is worth it mainly for **behaviour and format** a prompt can't reliably pin down, **latency/cost** (a small fine-tuned model replacing a big prompted one), or a **specialised style/skill** — never as the first thing you try.

# Part 1 · When to Fine-Tune (and When Not)

The most valuable fine-tuning skill is not fine-tuning when you shouldn't. It's expensive in data, compute, and maintenance, and it's the wrong fix for the problem people most often reach for it to solve. This chapter is the decision, and it's the most important one in the book.

## 1.1 The four levers, in order of cost

| Lever               | Changes                    | Cost     | Fixes                                              |
|---------------------|----------------------------|----------|----------------------------------------------------|
| <b>Prompting</b>    | the input                  | trivial  | instructions, format, simple behaviour             |
| <b>RAG</b>          | the input (adds facts)     | low      | missing/changing <i>knowledge</i>                  |
| <b>Fine-tuning</b>  | the weights                | high     | ingrained behaviour, style, format, a narrow skill |
| <b>Pre-training</b> | the weights (from scratch) | enormous | almost never you                                   |

Always climb from the top. Each rung down is more powerful and far more expensive to build and maintain.

## 1.2 The question that settles it: knowledge or behaviour?

The single most useful distinction:

- **Missing knowledge** ("it doesn't know our product / today's data") → **RAG**, not fine-tuning. Fine-tuning is a poor and expensive way to inject facts, and it goes stale the moment the facts change. Retrieval keeps facts fresh and citable.
- **Wrong behaviour, style, or format** ("it won't consistently output our JSON shape / house tone / follow this niche procedure," even with good prompts) → **fine-tuning** can help, by baking the pattern into the weights.

### ENGINEER'S MENTAL MODEL

Fine-tuning teaches a *skill or a style*; RAG supplies *facts*. Teaching the model your API's exact response format or your support team's tone is a skill — fine-tuning fits. Getting it to answer questions about this week's inventory is facts — retrieval fits. Trying to fine-tune facts in is like hard-coding config values into a compiled binary: it works until they change, then you rebuild.

## 1.3 Good reasons to fine-tune

- **Format/behaviour a prompt can't reliably pin down** — a strict output structure or procedure the model keeps drifting from despite good prompting.
- **Cost/latency** — a small fine-tuned model matching a big prompted model's quality on your narrow task, far cheaper and faster at scale.
- **Style/voice** — a consistent tone or domain idiom that's hard to specify but easy to *show* with examples.
- **A narrow, well-defined skill** — classification, extraction, or a transformation you have lots of examples of.
- **Prompt-length savings** — folding a huge, repeated system prompt / many few-shot examples into the weights.

## 1.4 When NOT to fine-tune

### WHEN NOT TO FINE-TUNE

Don't fine-tune when: the gap is **knowledge** (use RAG); you **haven't seriously tried prompting** (few-shot + a schema fixes more than people expect); you have **too few or low-quality examples** (hundreds of clean, consistent examples beat thousands of noisy ones — and both beat a rushed dataset); the task or data **changes often** (you'd retrain constantly); or you need **traceability** ("why did it say that?" — retrieval cites sources, a fine-tuned model can't). In all these, fine-tuning is slower, costlier, and worse than the cheaper lever.

### COMMON MISTAKE

Fine-tuning to add knowledge. It's the classic misuse: teams fine-tune on their docs hoping the model "learns" them, then find it hallucinates confidently, forgets older facts, and needs retraining whenever docs change — while a simple RAG setup would have been cheaper, fresher, and citable. If the problem is *what the model knows*, reach for retrieval; reserve fine-tuning for *how it behaves*.

### EXERCISE 1.1

Take three "the model isn't doing what I want" problems (yours or invented). For each, classify the gap as knowledge or behaviour, and pick the cheapest lever that fixes it: better prompt, RAG, or fine-tuning. Be strict — you'll likely find most are prompt or RAG problems. For any you'd genuinely fine-tune, write one sentence on why prompting and RAG can't get there.

# Part 2 · The Landscape: Full, PEFT, SFT & Preference

"Fine-tuning" is an umbrella over several very different techniques. Picking the right one — and knowing you almost always want the cheap one — saves money and confusion. This chapter is the map.

## 2.1 Two independent choices

Fine-tuning decisions split along two axes:

**How much of the model do you train?**

| Approach                | What trains              | Cost                      | Notes                              |
|-------------------------|--------------------------|---------------------------|------------------------------------|
| <b>Full fine-tuning</b> | every weight             | very high (GPU, memory)   | rarely necessary; risks forgetting |
| <b>PEFT / LoRA</b>      | a tiny added adapter     | low                       | the practical default (Part 5)     |
| <b>QLoRA</b>            | LoRA on a quantized base | lowest (one consumer GPU) | how most people fine-tune now      |

**What are you teaching — a task, or a preference?**

| Method                              | Data it needs              | Teaches                            |
|-------------------------------------|----------------------------|------------------------------------|
| <b>SFT</b> (supervised fine-tuning) | input → ideal output pairs | "do it <i>like this</i> " (Part 4) |
| <b>Preference tuning</b> (DPO/RLHF) | chosen vs rejected pairs   | "prefer this over that" (Part 6)   |

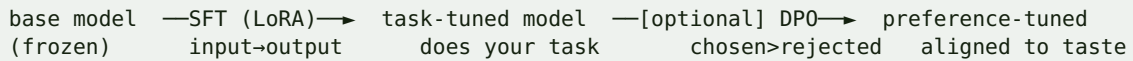
Most projects: **QLoRA + SFT**. You add a small adapter and train it on input→output examples. Preference tuning is a later refinement, not a starting point.

### ENGINEER'S MENTAL MODEL

Full fine-tuning rewrites the whole binary; **LoRA is a small patch loaded alongside the original** — the base model is untouched and the adapter is a few megabytes you can swap, version, or stack. SFT is "learn from worked examples"; preference tuning is "learn from which of two answers a human liked better." You almost always start with a LoRA adapter trained by SFT.

## 2.2 The typical pipeline

### DIAGRAM



Stage one (SFT) teaches the task and is enough for most needs. Stage two (preference tuning) nudges *style/quality* when SFT alone leaves it blunt — think "make it more helpful/on-brand," not "teach a new skill."

## 2.3 Open vs hosted fine-tuning

- **Open-weights models** (Llama, Mistral, Qwen, Gemma...) — you fine-tune them yourself with `transformers` / `peft` / `trl` on your own GPU, and you own the weights. Maximum control; you handle the infra. This book's main path.
- **Hosted fine-tuning APIs** — some providers let you upload a dataset and fine-tune their closed model via API. Zero infra, but a black box, ongoing per-token cost, and no portable weights.

### PRODUCTION TIP

Start with **QLoRA on a small open model** (7–8B) — it fits on a single consumer/cloud GPU, trains in hours not days, produces a tiny swappable adapter, and tells you fast whether fine-tuning even helps your task. Only scale to a bigger model or full fine-tuning if the small QLoRA run proves the approach but plateaus on quality. Prove the idea cheap before you spend.

### COMMON MISTAKE

Reaching for **full fine-tuning** by default. It needs far more GPU memory, risks **catastrophic forgetting** (the model loses general ability while learning your task), and produces a giant new checkpoint per experiment — for results LoRA usually matches at a fraction of the cost. Full fine-tuning is a last resort for when adapters genuinely can't reach the quality bar, not the starting point.

### EXERCISE 2.1

Map your intended fine-tune onto both axes: full vs PEFT, and SFT vs preference. Justify each choice in a sentence. For 95% of real tasks the honest answer is "QLoRA + SFT" — if yours isn't, articulate specifically why you need full fine-tuning or preference tuning, and what cheaper option you ruled out.

# Part 3 · Data Preparation — the Dataset *Is* the Product

Fine-tuning success is decided here, not in the training code. The model becomes its data: a clean, consistent dataset of a few hundred examples beats a sloppy one of tens of thousands. If you take one thing from this book, take this — **the dataset is the product, and everything else is plumbing.**

## 3.1 The format: chat / instruction pairs

Most fine-tuning data is a list of examples, each an input and the ideal output, in the model's chat format:

JSON

```
{
  "messages": [
    {
      "role": "system",
      "content": "You extract invoice fields as JSON."
    },
    {
      "role": "user",
      "content": "Invoice from Acme, total $340.50, due 2026-01-15"
    },
    {
      "role": "assistant",
      "content": '{"vendor": "Acme", "total": 340.50, "due": "2026-01-15"}'
    }
  ]
}
```

Each line is one training example. The `assistant` message is the **target** — the behaviour you're teaching. Every target must be exactly what you'd want the model to produce, because it will learn to imitate them faithfully, warts included.

### ENGINEER'S MENTAL MODEL

The dataset is your spec, expressed by example instead of by rule. The model doesn't learn what you *meant* — it learns what you *showed*. Inconsistent targets (three different JSON styles, varying tone) teach it to be inconsistent. Curating this data is the real engineering; treat it with the rigor you'd give production code, including review and version control.

## 3.2 Quality beats quantity

- **Consistency** — every example follows the same format, tone, and conventions. One diverging style teaches the model to diverge.
- **Correctness** — a wrong target teaches a wrong behaviour, confidently. Review the data.
- **Coverage** — include the variety and edge cases you expect at inference; the model generalises from what it's seen.
- **Quantity** — often **hundreds to a few thousand** excellent examples for a LoRA on a focused task. More helps only if it stays clean.

### 3.3 Splits and leakage

Hold out a **validation** and a **test** set the model never trains on, exactly as in any ML (the ML/DL books). Without them you can't tell learning from memorising, and you can't compare to the base model honestly.

#### COMMON MISTAKE

Garbage in, garbage baked in. Teams scrape or auto-generate a big dataset without reviewing it, then fine-tune on inconsistent, wrong, or duplicated examples — and the model faithfully learns the mess, often getting *worse* than the base. Spend most of your effort here: dedupe, review a real sample by hand, enforce one consistent target format, and fix or drop bad examples. A small clean dataset is the single biggest lever on the outcome.

#### PRODUCTION TIP

Keep a **held-out test set of real, hard cases** and version your dataset like code (it *is* the product). When you iterate — and you will — you need to prove a new dataset version beats the old one on the same fixed test set (Part 7). An unversioned dataset makes every "did that help?" question unanswerable.

#### WHEN NOT TO GENERATE SYNTHETIC DATA

Using a bigger LLM to generate training data is tempting and sometimes useful, but synthetic data inherits the generator's biases and errors, and can quietly cap your model at the teacher's quality (or teach its quirks). If you use it, **review it as strictly as human data** and mix in real examples — don't fine-tune on unvetted machine-generated data and assume it's clean.

#### EXERCISE 3.1

Build a small SFT dataset (100–300 examples) for a focused task in the chat format above, with one strict, consistent target format. Hand-review a random 20 and fix every inconsistency. Split into train/val/test. Then deliberately corrupt 20% of the training targets (vary the format), and note that you've just guaranteed an inconsistent model — the dataset is the behaviour.

# Part 4 · Supervised Fine-Tuning (SFT)

SFT is the core technique: continue training the model on your input→output examples so it learns to produce those outputs. It's the deep-learning training loop (your DL book) applied to a language model, and the Hugging Face **TRL** library wraps it into a few lines. This chapter is the mechanics; Part 5 makes it cheap with LoRA.

## 4.1 What SFT actually does

The model is trained on your examples with the same objective it was pretrained with — predict the next token — but now on *your* input→output pairs, with the loss computed on the **assistant's response** (the target). Do that over a few passes and the target behaviour settles into the weights.

### ENGINEER'S MENTAL MODEL

SFT is exactly the DL training loop from your Deep Learning book — forward, loss, backward, step — where the "labels" are your ideal responses and the loss is next-token prediction over them. Everything you learned there applies: learning rate matters most, watch train/val loss, overfitting is real, and epochs are few. A language model is just a (very large) network being trained on your data.

## 4.2 The minimal SFT run (TRL)

### PYTHON

```
from datasets import load_dataset
from trl import SFTTrainer, SFTConfig
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "meta-llama/Llama-3.1-8B-Instruct"      # any open instruct model; check
the current id
tok = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id, device_map="auto")

data = load_dataset("json", data_files={"train": "train.jsonl", "test": "val.jsonl"})

trainer = SFTTrainer(
    model=model,
    args=SFTConfig(
        output_dir="out", num_train_epochs=2,          # few epochs – 1–3 typical
        per_device_train_batch_size=2, gradient_accumulation_steps=8,
        learning_rate=2e-5, bf16=True, logging_steps=10, eval_strategy="epoch",
    ),
    train_dataset=data["train"], eval_dataset=data["test"],
```

```
)
trainer.train()
trainer.save_model("out/final")
```

TRL handles applying the chat template, masking the loss to the response, and the loop. (In Part 5 you add a few lines to make this a LoRA run so it fits a single GPU.)

## 4.3 The knobs that matter

- **Epochs** — usually **1-3**. LLMs overfit fast on small data; more epochs memorises and degrades general ability.
- **Learning rate** — small ( $\approx 1e-5$ – $2e-5$  full; higher for LoRA). Too high erases pretrained ability (catastrophic forgetting).
- **Batch size / gradient accumulation** — accumulate to simulate a larger batch when GPU memory is tight.
- **Sequence length** — long enough for your examples; longer costs more memory.

## 4.4 Watch the loss, but trust the eval

Falling training loss means it's learning your data — not that it got *better* at the task or kept its general skills. Only a proper evaluation (Part 7) on held-out cases tells you that. Loss is a training instrument; eval is the verdict.

### COMMON MISTAKE

Training for too many epochs. On a small dataset the training loss keeps dropping and it feels like progress, but the model is **memorising** the examples and **forgetting** its general abilities — it aces your training set and gets worse everywhere else. Start with 1-3 epochs, watch validation loss for the uptick (your DL overfitting signal), and stop early.

### PRODUCTION TIP

Before a long run, **overfit a tiny subset on purpose** (10-20 examples, a few steps) and confirm the model can reproduce them — the same single-batch sanity check from the DL book. If it can't, your data format, chat template, or loss masking is wrong, and you've caught it in minutes instead of after a multi-hour run on the full set.

### EXERCISE 4.1

Run SFT on your Part 3 dataset with a small open model (use the Part 5 LoRA config so it fits your GPU). First do the tiny-subset overfit test. Train 2 epochs, watch train/val loss, and generate on a few held-out inputs. Then train for 10 epochs and observe the model memorising and degrading — see overfitting in a language model directly.

# Part 5 · LoRA & QLoRA — Fine-Tuning You Can Afford

Full fine-tuning updates every weight — billions of them — needing huge GPU memory and producing a giant checkpoint per run. **LoRA** makes this practical: freeze the whole base model and train a tiny added adapter instead. **QLoRA** goes further and lets you fine-tune a big model on a single consumer GPU. This is how almost everyone fine-tunes now.

## 5.1 The LoRA idea

Instead of changing the model's weight matrices, LoRA **freezes them** and learns a small pair of low-rank matrices *alongside* each — a tiny "diff" added at inference. You train ~0.1–1% as many parameters, with almost no quality loss on most tasks.

### DIAGRAM

|          |                                                                            |
|----------|----------------------------------------------------------------------------|
| full FT: | update W (millions/billions of numbers per layer)                          |
| LoRA:    | freeze W, learn a small A·B added to it (a few thousand) → $W + A \cdot B$ |

The result is a **few-megabyte adapter**, not a multi-gigabyte model.

### ENGINEER'S MENTAL MODEL

LoRA is a diff/patch against a read-only base. The huge base weights never change (so no catastrophic forgetting, and one base serves many adapters); your training only touches the small patch. That's why LoRA adapters are tiny, swappable, and cheap to store and version — you can keep a dozen task-specific adapters for one base model and load whichever you need.

## 5.2 QLoRA: LoRA on a quantized base

QLoRA loads the frozen base model in **4-bit** (quantized) to slash memory, then trains LoRA adapters on top in full precision. The base being 4-bit costs a little quality but means an 8B–13B model fine-tunes on **one 16-24GB GPU** — the reason fine-tuning is now within reach without a cluster.

## 5.3 LoRA/QLoRA in code (peft + trl)

Add a `LoraConfig` to the SFT run from Part 4:

### PYTHON

```
from peft import LoraConfig
from transformers import BitsAndBytesConfig
```

```
# QLoRA: load the base in 4-bit
bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                        bnb_4bit_compute_dtype="bfloat16")
model = AutoModelForCausalLM.from_pretrained(model_id, quantization_config=bnb,
                                             device_map="auto")

lora = LoraConfig(
    r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules="all-linear",          # which layers get adapters
    task_type="CAUSAL_LM",
)
# pass peft_config=lora to SFTTrainer – that's the whole change from Part 4
trainer = SFTTrainer(model=model, args=cfg, train_dataset=..., peft_config=lora)
trainer.train()
trainer.save_model("out/adapter")        # saves the tiny adapter, not the base
```

The one new hyperparameter to know is `r` (the adapter rank): higher = more capacity and more parameters (8–32 is a common range); start around 16.

## 5.4 Adapters are swappable

Because the base is untouched, you can keep **many adapters** for one base model and load the one you need at inference — a support-tone adapter, an extraction adapter, a summarisation adapter — all sharing one set of base weights in memory.

### PRODUCTION TIP

**Default to QLoRA.** It fits a real model on one affordable GPU, trains in hours, produces a tiny adapter you can version in Git-LFS, avoids catastrophic forgetting by construction, and matches full fine-tuning on most tasks. Only consider full fine-tuning if a well-tuned QLoRA run (try higher `r` first) provably can't reach your quality bar.

### COMMON MISTAKE

Setting the LoRA learning rate as low as a full-fine-tune LR and seeing nothing happen. LoRA trains far fewer parameters and wants a **higher** learning rate (often 1e-4–2e-4, vs ~2e-5 for full). If your LoRA "isn't learning," the LR is usually too low — or `target_modules` is missing the layers that matter (use `all-linear` to be safe).

### EXERCISE 5.1

Convert your Part 4 SFT run to QLoRA: 4-bit base + a `LoraConfig` with `r=16` and a LoRA-appropriate learning rate. Confirm it fits your GPU and check the trainable-parameter count (`model.print_trainable_parameters()`) — you'll see <1% of the weights train. Save the adapter and note its size (megabytes). Bonus: train a second adapter for a different task on the same base.

# Part 6 · Preference Tuning (DPO & RLHF)

SFT teaches the model to imitate good examples. **Preference tuning** teaches it something SFT can't easily express: to prefer *better* answers over *worse* ones, from pairs a human (or model) ranked. It's how base models become the polished "assistants" you use — and for most fine-tuning projects it's an optional second stage, not a starting point.

## 6.1 SFT vs preference — the difference

- **SFT** learns from **one right answer** per input ("produce this").
- **Preference tuning** learns from **a comparison** — two answers to the same prompt, one labelled *chosen*, one *rejected* ("this one is better than that one").

Comparisons capture fuzzy quality — helpfulness, tone, safety, "feels right" — that's hard to write as a single gold target but easy to judge between two options.

JSON

```
{
  "prompt": "Explain a JWT to a beginner.",
  "chosen": "A JWT is a signed token that proves who you are...",
  "rejected": "JWT stands for JSON Web Token. It has three base64 parts..."
}
```

### ENGINEER'S MENTAL MODEL

SFT is learning from a worked solution; preference tuning is learning from code review — "this version is better than that one, here's the pair." When "good" is a matter of taste or degree you can *recognise* but not fully *specify*, showing the model ranked comparisons teaches it the gradient of quality, not just one acceptable point.

## 6.2 RLHF vs DPO

- **RLHF** (Reinforcement Learning from Human Feedback) — the original: train a separate *reward model* on the preferences, then use RL (PPO — your RL book) to optimise the LLM against it. Powerful, but a complex, finicky multi-stage pipeline.
- **DPO** (Direct Preference Optimization) — the modern default: skips the reward model and RL loop, and optimises the model directly on the chosen/rejected pairs with a simple loss. Far simpler, stable, and usually what you should use.

PYTHON

```
from trl import DPOTrainer, DPOConfig
trainer = DPOTrainer(model=model, args=DPOConfig(output_dir="dpo", beta=0.1),
                    train_dataset=pref_data, peft_config=lora) # LoRA works here
```

```
too
trainer.train()
```

`beta` controls how far the tuned model may drift from the reference — low keeps it close, higher lets it change more.

## 6.3 The usual order: SFT first, then DPO

Preference tuning refines a model that can already do the task. The standard recipe is **SFT to teach the skill/format, then optionally DPO to polish quality and style**. DPO on a base that hasn't learned the task first tends to disappoint.

### WHEN NOT TO DO PREFERENCE TUNING

Skip it unless SFT has genuinely plateaued on *quality* (not capability) and you have real preference pairs. It needs comparison data (expensive to collect well), adds a training stage, and fixes "which good answer is better" — not "can't do the task" (that's SFT) or "doesn't know the facts" (that's RAG). Most projects ship on SFT alone; DPO is the finishing polish, not the foundation.

### COMMON MISTAKE

Jumping to RLHF/DPO to fix a model that simply hasn't learned the task yet. Preference tuning sharpens *preferences* between already-plausible answers; if the model can't produce a decent answer at all, no amount of ranking pairs will teach it the skill — that's SFT's job, and better/more data. Get the task right with SFT first; only then consider polishing with DPO.

### EXERCISE 6.1

Starting from your SFT model (Part 5), build a small preference dataset: for ~50 prompts, generate two responses and label the better one (by a rubric, or a stronger model as judge). Run DPO with LoRA on top of the SFT adapter and compare outputs before/after on held-out prompts. Judge honestly whether the polish was worth the extra stage — or whether better SFT data would've done more.

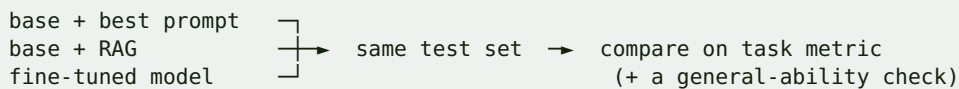
# Part 7 · Evaluating a Fine-Tuned Model

The question that matters is not "did the loss go down?" but "**is it actually better than what I started with, without breaking anything?**" Fine-tuning can improve your task while quietly degrading general ability, so evaluation is what tells you whether the whole exercise was worth it. Without it, you're shipping on faith.

## 7.1 Always compare against the baseline

Every fine-tune must be measured against the **base model with your best prompt** on the same held-out test set. That's the bar to beat — often fine-tuning's real competitor is a better prompt, and only a head-to-head reveals whether the training earned its cost.

### DIAGRAM



### ENGINEER'S MENTAL MODEL

This is A/B testing your model versions (your Data Analysis book): a fixed test set, the same metric, the candidates side by side. "The fine-tune feels better" is not a result — a measured win on held-out cases is. And the baseline isn't "nothing," it's the *cheapest lever done well*: base + a good prompt. Beat that, or don't ship the fine-tune.

## 7.2 What to measure

- **Task metric** — accuracy, exact-match, F1, or a rubric score on held-out cases: did it get better at the thing you tuned for?
- **General ability (regression check)** — a small suite of *general* prompts to catch catastrophic forgetting. A model that nails your task but got dumber elsewhere may be a net loss.
- **Format/behaviour adherence** — if you tuned for a strict output shape, measure the compliance rate programmatically.
- **Cost/latency** — often the *point* of fine-tuning (small tuned model vs big prompted one); measure it.

## 7.3 How to score

- **Programmatic checks** where possible — exact match, schema validity, does the output parse/compile. Cheap, deterministic, trustworthy.
- **LLM-as-judge** for open-ended quality — a stronger model grades responses against a rubric (your Agents book); validate the judge against some human ratings.
- **Human review** on a sample for anything high-stakes — the ground truth the others approximate.

### COMMON MISTAKE

Declaring success from the training loss and a few cherry-picked generations. Falling loss only means the model fit your training data; it says nothing about held-out performance or what the model *lost*. Teams ship a fine-tune that memorised the train set and forgot how to do everything else. Measure on a real held-out set, against the baseline, with a general-ability regression check — every time.

### PRODUCTION TIP

Keep a fixed **evaluation harness** — a test set plus scoring — and run *every* candidate (each dataset version, base model, and hyperparameter set) through it, logging the numbers. Fine-tuning is deeply iterative; without a stable scoreboard you can't tell whether change #7 actually beat change #3, and you'll go in circles chasing vibes.

### WHEN NOT TO TRUST A SINGLE METRIC

One aggregate score hides regressions. A model can lift average task accuracy while getting much worse on an important sub-case, or gain on format while losing reasoning. Break results down by category (your Data Analysis segmenting habit), and always pair the task metric with the general-ability check — never ship on a lone number.

### EXERCISE 7.1

Build an eval harness for your fine-tune: a held-out test set with programmatic task scoring plus a 10-prompt general-ability regression check. Score three candidates on it — (1) base + best prompt, (2) your SFT model, (3) base + RAG if applicable — and produce a table. Decide from the numbers whether the fine-tune actually beat the cheapest lever, and whether it lost any general ability.

# Part 8 · Serving a Fine-Tuned Model

A trained adapter sitting in a folder helps nobody. Serving a fine-tuned model has a few wrinkles beyond serving any model — adapters, quantization, and picking an inference engine — but it's squarely your backend territory. This chapter gets it to an endpoint.

## 8.1 Adapter: load-on-top or merge

Your LoRA training produced a small adapter, not a full model. Two ways to serve it:

- **Load base + adapter at runtime** — load the base model, then apply the adapter. Keeps the adapter swappable and lets one base serve several adapters.
- **Merge** the adapter into the base once ( `merge_and_unload()` ) to produce a standalone model — simpler to deploy, slightly faster, but no longer swappable.

PYTHON

```
from peft import PeftModel
from transformers import AutoModelForCausalLM

base = AutoModelForCausalLM.from_pretrained(base_id, device_map="auto")
model = PeftModel.from_pretrained(base, "out/adapter") # base + adapter
merged = model.merge_and_unload() # optional: bake it in
merged.save_pretrained("merged-model")
```

### ENGINEER'S MENTAL MODEL

Loading the adapter at runtime is dynamic linking — one shared base in memory, small adapters loaded per task; merging is static linking into one binary. Choose dynamic when you serve **many** adapters off one base (memory-efficient, swappable), static when you deploy **one** fine-tuned model and want the simplest, fastest artifact.

## 8.2 Use a real inference engine

Don't serve production LLM traffic with a bare `transformers generate()` loop — it's slow and won't batch well. Use a purpose-built server:

- **vLLM** — high-throughput serving with continuous batching; the common default, and it can serve LoRA adapters directly.
- **TGI** (Text Generation Inference) — Hugging Face's production server.
- **Ollama / llama.cpp** — great for local and quantized (GGUF) serving.

Most expose an **OpenAI-compatible API**, so your existing client code (Python book) points at your endpoint by changing the `base_url` — the same trick as serving local models.

## 8.3 Quantize for inference too

Just as QLoRA quantized the base for *training*, you can quantize the merged model for *serving* (GPTQ, AWQ, GGUF) to cut memory and cost, usually with minor quality loss. Often the difference between fitting on an expensive GPU and an affordable one.

### PRODUCTION TIP

Serve with **vLLM behind an OpenAI-compatible endpoint**, then your app talks to your fine-tuned model with the *exact* client code it used for the hosted API — only `base_url` changes. This keeps prompting/agents/RAG code identical whether you're on a hosted model or your own fine-tune, so switching is a config change, not a rewrite.

### COMMON MISTAKE

Serving base + adapter but loading the **wrong base** (or a differently quantized/precision base than you trained against), then getting garbled or degraded output. The adapter is a diff computed against a *specific* base — pin the exact base model id and precision, and match the chat template and tokenizer you fine-tuned with. Adapter/base mismatch is the top "it worked in training, breaks in serving" bug.

### WHEN NOT TO SELF-HOST

If you used a **hosted** fine-tuning API, serving is handled for you — don't build infra you don't need. And even for an open-weights fine-tune, if traffic is low and infra cost/effort is high, a managed inference host may be cheaper than running your own GPU. Self-host for scale, cost-at-volume, privacy, or control — not by default.

### EXERCISE 8.1

Serve your fine-tuned adapter two ways: (1) base + adapter loaded at runtime, and (2) merged into a standalone model. Put each behind an OpenAI-compatible endpoint (vLLM or Ollama) and confirm your Python LLM client works against it by changing only `base_url`. Compare load time, memory, and tokens/sec. Then quantize the merged model and re-measure.

# Part 9 · Fine-Tuning in Production

Fine-tuning isn't a one-shot event; it's an iterative pipeline you'll run many times as your data and needs evolve. This chapter covers the realities that decide whether a fine-tune stays valuable: cost, iteration discipline, versioning, and the ongoing maintenance a frozen prompt never demands.

## 9.1 The real cost is iteration and maintenance

The GPU bill for one QLoRA run is modest. The real cost is everything around it: curating and re-curating data, running experiments, evaluating, and — the part teams forget — **maintaining the model over time** as requirements drift and base models improve. A prompt you can change in seconds; a fine-tune you re-train and re-evaluate.

### ENGINEER'S MENTAL MODEL

A fine-tuned model is a build artifact with a supply chain: data → training → eval → serving, each versioned and reproducible. Treat it like a service you own, not a file you made once — it has a lifecycle, it drifts, and it needs an owner. The prompt-and-RAG alternative has almost no supply chain, which is a big part of why it wins when it can.

## 9.2 Version everything, reproducibly

To answer "why does the deployed model behave like this?" and "did the last change help?", version the whole chain:

- **The dataset** (it's the product — Part 3), with a fixed test set.
- **The base model** id and precision (the adapter is meaningless without it — Part 8).
- **The training config** (hyperparameters, seed).
- **The resulting adapter/model**, tagged, with its eval scores attached.

Track experiments (MLflow / Weights & Biases, per the ML book) so every run's data version, config, and metrics are recorded.

## 9.3 The iteration loop

### DIAGRAM

```

curate data → train (QLoRA/SFT) → evaluate vs baseline → ship or iterate
      ↑                               |
      └── most gains come from better DATA, not more training ─┘
  
```

When results disappoint, the highest-leverage fix is almost always **the data** (more, cleaner, better-covered examples), not more epochs or a fancier method.

## 9.4 Base models move under you

Open models improve fast. A fine-tune that beat the base six months ago may be worse than *today's* base model with a good prompt. Periodically re-run your eval harness (Part 7) with the latest base + prompt as a fresh baseline, and be willing to **retire a fine-tune** when the frontier catches up. Sunk cost is not a reason to keep serving a stale adapter.

### COMMON MISTAKE

Fine-tuning once and treating it as done. The model silently rots: the base model's successors get better, your data distribution shifts, and requirements change — while your frozen adapter stays put. Re-baseline periodically, keep the eval harness runnable, and own the model's lifecycle. A fine-tune is a living dependency, not a finished file.

### WHEN NOT TO KEEP FINE-TUNING

If each iteration's eval barely moves, or a newer base model with a good prompt now matches your fine-tune, **stop** — you've hit the point where more tuning isn't the lever. Put the effort into data quality, or step back to prompting/ RAG on a stronger base. Knowing when to quit fine-tuning is as valuable as knowing when to start.

### PRODUCTION TIP

Keep the **base + best-prompt baseline wired into your eval harness permanently**, and re-run it whenever a notable new base model ships. It's your early-warning system for "the fine-tune is no longer worth it" — the single check that stops you maintaining an adapter the frontier has already made obsolete.

### EXERCISE 9.1

Set up a reproducible fine-tune: version your dataset, pin the base id + config, and log the run and its eval scores to an experiment tracker. Do two iterations that change *only the data* (add/clean examples) and confirm from the tracked metrics whether data quality moved the needle. Record the base+prompt baseline in the same tracker as your obsolescence check.

# Part 10 · Capstone & Cheat Sheet

## 10.1 Capstone — fine-tune a model, honestly

Pick a narrow, behaviour/format task (not a knowledge task) and go end to end:

1. **Justify it** — confirm the gap is behaviour/format, not knowledge, and that a good prompt and RAG can't get there (Part 1). Write the base+prompt baseline you must beat.
2. **Data** — curate a few hundred clean, consistent chat examples with one strict target format; split train/val/test (Part 3).
3. **Train** — QLoRA + SFT on a small open model; do the tiny-subset overfit test first; 1-3 epochs (Parts 4-5).
4. **Evaluate** — score on the held-out set against base+prompt, with a general-ability regression check; break results down by category (Part 7).
5. **(Optional) polish** — DPO on preference pairs only if SFT plateaued on quality (Part 6).
6. **Serve** — base+adapter or merged, behind an OpenAI-compatible endpoint (vLLM), pinning the exact base (Part 8).
7. **Operate** — version data/config/adapter+scores; keep the baseline wired in to catch obsolescence (Part 9).

Success = a measured win over base+prompt on your task, no general-ability regression, at acceptable cost. If you can't beat the prompt, that's a real (and common) result: ship the prompt.

## 10.2 Decision cheat sheet

### DIAGRAM

|                                                                  |                                           |
|------------------------------------------------------------------|-------------------------------------------|
| Problem with the model?                                          |                                           |
| missing/changing facts?                                          | → RAG (not fine-tuning)                   |
| simple instruction/format?                                       | → better prompt (few-shot + schema)       |
| behaviour/style/format a prompt can't pin down, OR cost/latency? | → fine-tune                               |
| need "why did it say that?"                                      | → RAG (citable), not fine-tuning          |
| Which fine-tune?                                                 |                                           |
| default                                                          | → QLoRA + SFT (small open model)          |
| SFT plateaued on quality/taste                                   | → + DPO (needs preference pairs)          |
| adapters can't reach the bar                                     | → consider full fine-tuning (last resort) |
| results disappoint                                               | → fix the DATA first, not the epochs      |

## 10.3 Concept cheat sheet

### DIAGRAM

|       |                                                          |
|-------|----------------------------------------------------------|
| SFT   | supervised fine-tuning: learn from input-output pairs    |
| LoRA  | freeze base, train a tiny added adapter (~MB, swappable) |
| QLoRA | LoRA on a 4-bit base → fine-tune big models on one GPU   |

|                         |                                                             |
|-------------------------|-------------------------------------------------------------|
| r (rank)                | LoRA adapter capacity; start ~16                            |
| epochs                  | 1–3; more memorises and forgets                             |
| LR                      | LoRA higher (~1e-4–2e-4); full lower (~2e-5)                |
| DPO                     | preference tuning without a reward model/RL (use over RLHF) |
| merge                   | bake adapter into base → standalone model (static link)     |
| catastrophic forgetting | losing general ability while learning your task             |

## 10.4 The five rules

1. **Try the cheaper levers first.** Prompt, then RAG, then fine-tune. Most teams never need to fine-tune.
2. **Knowledge → RAG; behaviour/style → fine-tune.** Never fine-tune to inject facts.
3. **The dataset is the product.** A few hundred clean, consistent examples beat thousands of noisy ones; fix data before epochs.
4. **QLoRA + SFT is the default.** Tiny swappable adapter, one GPU, no forgetting; full fine-tuning is a last resort.
5. **Prove it beats base + best prompt,** with no general-ability regression, or don't ship it — and re-check as base models improve.

### YOU'RE READY

You can now tell when fine-tuning is the right lever (rarely) and when prompting or RAG wins (usually), curate the dataset that actually determines the outcome, run QLoRA + SFT and — when needed — DPO, evaluate honestly against the cheapest baseline, and serve and maintain the result. The discipline of reaching for the *smallest* change that solves the problem — and being willing to conclude "the prompt was enough" — is what separates an engineer who fine-tunes well from one who fine-tunes reflexively. Change the weights only when the context can't.